

```

1 def print_tableau(tableau, basic_vars, variables):
2     header = f"{'Basic':<8}" + "".join([f"{'v':<8}" for v in variables]) + f"{'RHS':<8}"
3     print("-" * len(header))
4     print(header)
5     print("-" * len(header))
6     for i in range(len(basic_vars)):
7         row_str = f"{'variables[basic_vars[i]]':<8}"
8         row_str += "".join([f"{'val':<8.2f}" for val in tableau[i][:-1]])
9         row_str += f"{'tableau[i][-1]:<8.2f}"
10        print(row_str)
11
12    z_row = f"{'Z-Row':<8}"
13    z_row += "".join([f"{'val':<8.2f}" for val in tableau[-1][:-1]])
14    z_row += f"{'tableau[-1][-1]:<8.2f}"
15    print(z_row)
16    print("-" * len(header) + "\n")
17
18 def big_m_simplex():
19     print("=== BIG-M SIMPLEX METHOD ===")
20     variables = ['x1', 'x2', 's1', 's2', 'A1']
21     M = 10000.0
22     tableau = [
23         [1.0, 1.0, -1.0, 0.0, 1.0, 10.0],
24         [2.0, 1.0, 0.0, 1.0, 0.0, 16.0],
25         [4.0 - M, 8.0 - M, M, 0.0, 0.0, -10.0 * M]
26     ]
27     basic_vars = [4, 3]
28     iteration = 0
29     while True:
30         print(f"Iteration {iteration}:")
31         print_tableau(tableau, basic_vars, variables)
32
33         z_row = tableau[-1][:-1]
34
35         if all(coeff >= -1e-5 for coeff in z_row):
36             print("Optimality reached! All Z-row coefficients are >= 0.\n")
37             break
38
39         enter_idx = z_row.index(min(z_row))
40         print(f"Entering Variable: {variables[enter_idx]}")
41
42         ratios = []
43         for i in range(len(basic_vars)):
44             if tableau[i][enter_idx] > 0:
45                 ratios.append(tableau[i][-1] / tableau[i][enter_idx])
46             else:
47                 ratios.append(float('inf'))
48
49         if all(r == float('inf') for r in ratios):
50             print("Problem is unbounded.")
51             return
52
53         leave_idx = ratios.index(min(ratios))
54         print(f"Leaving Variable: {variables[basic_vars[leave_idx]]}")
55
56         pivot = tableau[leave_idx][enter_idx]

```

```

56     pivot = tableau[leave_idx][enter_idx]
57     print(f"Pivot Element: {pivot:.2f}\n")
58
59     basic_vars[leave_idx] = enter_idx
60
61     tableau[leave_idx] = [val / pivot for val in tableau[leave_idx]]
62
63     for i in range(len(tableau)):
64         if i != leave_idx:
65             factor = tableau[i][enter_idx]
66             tableau[i] = [tableau[i][j] - factor * tableau[leave_idx][j] for j in range(len(tableau[i]))]
67
68     iteration += 1
69
70     print("=== FINAL SOLUTION ===")
71     solution = {var: 0 for var in variables}
72     for i, b_var in enumerate(basic_vars):
73         solution[variables[b_var]] = tableau[i][-1]
74
75     for var, val in solution.items():
76         print(f"{var} = {val:.2f}")
77
78     optimal_z_prime = tableau[-1][-1]
79     optimal_cost = -optimal_z_prime
80     print(f"\nMinimum Optimal Cost = {optimal_cost:.2f}")
81
82 if __name__ == "__main__":
83     big_m_simplex()

```

```
PS C:\Users\rajiv\OneDrive\Desktop\OT 2026 materials-20260820T135936Z-1-001\OT_github\OT1> python -u "c:\Users\rajiv\OneDrive\Desktop\OT 2026 materials-20260820T135936Z-1-001\OT_github\OT1\Modi_M.py"
=== RUNNING VAM ===
Initial Cost (VAM) = 145

=== RUNNING MODI (OPTIMIZATION) ===
Negative Delta found at (3, 3). Delta = -2
Shifting theta = 5 units along closed loop.

Optimality Condition Met! All Deltas >= 0.

=== FINAL ALLOCATIONS ===
[0, 15, 0, 0]
[10, 0, 15, 0]
[0, 0, 0, 10]
[0, 0, 0, 5]

Final Minimum Transportation Cost = 135
Verification: sum(allocation[i][j] * cost[i][j]) validates this total.
```

```

1 import sys
2 import copy
3
4 def calculate_penalties(costs, supply, demand, r_done, c_done):
5     r_pen = []
6     for i in range(len(supply)):
7         if r_done[i]:
8             r_pen.append(-1)
9             continue
10        row = [costs[i][j] for j in range(len(demand)) if not c_done[j]]
11        if len(row) >= 2:
12            s_row = sorted(row)
13            r_pen.append(s_row[1] - s_row[0])
14        elif len(row) == 1:
15            r_pen.append(row[0])
16        else:
17            r_pen.append(-1)
18
19    c_pen = []
20    for j in range(len(demand)):
21        if c_done[j]:
22            c_pen.append(-1)
23            continue
24        col = [costs[i][j] for i in range(len(supply)) if not r_done[i]]
25        if len(col) >= 2:
26            s_col = sorted(col)
27            c_pen.append(s_col[1] - s_col[0])
28        elif len(col) == 1:
29            c_pen.append(col[0])
30        else:
31            c_pen.append(-1)
32
33    return r_pen, c_pen
34
35 def vogel_approximation(costs, supply, demand):
36     print("=== RUNNING VAM ===")
37     allocations = [[0]*len(demand) for _ in range(len(supply))]
38     r_done = [False]*len(supply)
39     c_done = [False]*len(demand)
40
41     basic_cells = []
42     sup = copy.deepcopy(supply)
43     dem = copy.deepcopy(demand)
44
45     while not (all(r_done) or all(c_done)):
46         r_pen, c_pen = calculate_penalties(costs, sup, dem, r_done, c_done)
47
48         max_r = max(r_pen) if r_pen else -1
49         max_c = max(c_pen) if c_pen else -1
50
51         if max_r == -1 and max_c == -1:
52             break
53
54         if max_r >= max_c:
55             i = r_pen.index(max_r)
56             valid_costs = [(costs[i][j], j) for j in range(len(dem)) if not c_done[j]]
57             j = min(valid_costs, key=lambda x: x[0])[1]

```

```

56     valid_costs = [(costs[i][j], j) for j in range(len(dem)) if not c_done[j]]
57     j = min(valid_costs, key=lambda x: x[0])[1]
58     else:
59         j = c_pen.index(max_c)
60         valid_costs = [(costs[i][j], i) for i in range(len(sup)) if not r_done[i]]
61         i = min(valid_costs, key=lambda x: x[0])[1]

```

```

62
63     qty = min(sup[i], dem[j])
64     allocations[i][j] = qty
65     basic_cells.append((i, j))
66     sup[i] -= qty
67     dem[j] -= qty

```

```

68
69     if sup[i] == 0 and dem[j] == 0:
70         c_done[j] = True
71     elif sup[i] == 0:
72         r_done[i] = True
73     else:
74         c_done[j] = True

```

```

75
76     req_cells = len(supply) + len(demand) - 1
77     if len(basic_cells) < req_cells:
78         print("Degeneracy detected in VAM. Adding 0 allocations.")
79         for i in range(len(supply)):
80             for j in range(len(demand)):
81                 if (i, j) not in basic_cells and len(basic_cells) < req_cells:
82                     allocations[i][j] = 0
83                     basic_cells.append((i, j))
84
85     return allocations, basic_cells

```

```

86
87 def calculate_potentials(costs, basic_cells, rows, cols):

```

```

88     u = [None] * rows
89     v = [None] * cols
90     u[0] = 0
91
92     progress = True
93     while progress and (None in u or None in v):
94         progress = False
95         for i, j in basic_cells:
96             if u[i] is not None and v[j] is None:
97                 v[j] = costs[i][j] - u[i]
98                 progress = True
99             elif v[j] is not None and u[i] is None:
100                 u[i] = costs[i][j] - v[j]
101                 progress = True
102     for i in range(rows):
103         if u[i] is None: u[i] = 0
104     for j in range(cols):
105         if v[j] is None: v[j] = 0
106
107     return u, v

```

```

108
109 def find_closed_loop(start_cell, basic_cells, rows, cols):

```

```

110     def dfs(current_cell, path, is_row_search):
111         if len(path) > 3 and path[-1] == start_cell:
112             return path

```

```

114 i, j = current_cell
115 search_space = basic_cells + [start_cell]
116
117 for r, c in search_space:
118     if (r, c) != current_cell and (r, c) not in path[1:]:
119         if (is_row_search and r == i) or (not is_row_search and c == j):
120             res = dfs((r, c), path + [(r, c)], not is_row_search)
121             if res: return res
122 return None
123
124 return dfs(start_cell, [start_cell], True)
125
126 def modi(costs, allocations, basic_cells):
127     print("\n=== RUNNING MODI (OPTIMIZATION) ===")
128     rows, cols = len(costs), len(costs[0])
129
130     while True:
131         u, v = calculate_potentials(costs, basic_cells, rows, cols)
132
133         min_delta = 0
134         enter_cell = None
135
136         for i in range(rows):
137             for j in range(cols):
138                 if (i, j) not in basic_cells:
139                     delta = costs[i][j] - (u[i] + v[j])
140                     if delta < min_delta:
141                         min_delta = delta
142                         enter_cell = (i, j)
143
144         if min_delta >= 0:
145             print("Optimality Condition Met! All Deltas >= 0.")
146             break
147
148         print(f"Negative Delta found at {enter_cell}. Delta = {min_delta}")
149         loop = find_closed_loop(enter_cell, basic_cells, rows, cols)
150         loop = loop[:-1]
151
152         minus_cells = loop[1:2]
153         theta = min(allocations[r][c] for r, c in minus_cells)
154         print(f"Shifting theta = {theta} units along closed loop.\n")
155
156         for idx, (r, c) in enumerate(loop):
157             if idx % 2 == 0:
158                 allocations[r][c] += theta
159             else:
160                 allocations[r][c] -= theta
161
162         basic_cells.append(enter_cell)
163         for r, c in minus_cells:
164             if allocations[r][c] == 0:
165                 basic_cells.remove((r, c))
166                 break
167
168

```

```

168
169     return allocations
170
171 def calculate_cost(costs, allocations):
172     total = 0
173     for i in range(len(costs)):
174         for j in range(len(costs[0])):
175             if allocations[i][j] > 0:
176                 total += costs[i][j] * allocations[i][j]
177     return total
178
179 if __name__ == "__main__":
180     costs = [
181         [5, 2, 4, 3],
182         [4, 8, 1, 6],
183         [4, 6, 7, 5],
184         [0, 0, 0, 0]
185     ]
186     supply = [15, 25, 10, 5]
187     demand = [10, 15, 15, 15]
188     allocations, basic_cells = vogel_approximation(costs, supply, demand)
189     initial_cost = calculate_cost(costs, allocations)
190     print(f"Initial Cost (VAM) = {initial_cost}")
191     final_alloc = modi(costs, allocations, basic_cells)
192     final_cost = calculate_cost(costs, final_alloc)
193
194     print("\n=== FINAL ALLOCATIONS ===")
195     for i in range(len(final_alloc)):
196         print(final_alloc[i])
197     print(f"\nFinal Minimum Transportation Cost = {final_cost}")
198     print("Verification: sum(allocation[i][j] * cost[i][j]) validates this")

```

```
PS C:\Users\rajiv\OneDrive\Desktop\OT 2026 materials-20260820T135936Z-1-001\OT_github\OT1> python -u "c:\Users\rajiv\OneDrive\Desktop\OT 2026 materials-20260820T135936Z-1-001\OT_github\OT1\BigM.py"
```

```
=== BIG-M SIMPLEX METHOD ===  
Iteration 0:
```

Basic	x1	x2	s1	s2	A1	RHS
A1	1.00	1.00	-1.00	0.00	1.00	10.00
s2	2.00	1.00	0.00	1.00	0.00	15.00
Z-Row	-9996.00	9992.00	10000.00	0.00	0.00	-100000.00

```
Entering Variable: x1  
Leaving Variable: s2  
Pivot Element: 2.00
```

```
Iteration 1:
```

Basic	x1	x2	s1	s2	A1	RHS
A1	0.00	0.50	-1.00	-0.50	1.00	2.00
x1	1.00	0.50	0.00	0.50	0.00	8.00
Z-Row	0.00	-4994.00	10000.00	4998.00	0.00	-20032.00

```
Entering Variable: x2  
Leaving Variable: A1  
Pivot Element: 0.50
```

```
Iteration 2:
```

Basic	x1	x2	s1	s2	A1	RHS
x2	0.00	1.00	-2.00	-1.00	2.00	4.00
x1	1.00	0.00	1.00	1.00	-1.00	6.00
Z-Row	0.00	0.00	12.00	4.00	9988.00	-56.00

```
Optimality reached! All Z-row coefficients are >= 0.
```

```
=== FINAL SOLUTION ===
```

```
x1 = 6.00  
x2 = 4.00  
s1 = 0.00  
s2 = 0.00  
A1 = 0.00
```

```
Minimum Optimal Cost = 56.00
```

```
PS C:\Users\rajiv\OneDrive\Desktop\OT 2026 materials-20260820T135936Z-1-001\OT_github\OT1> python -u "c:\Users\rajiv\OneDrive\Desktop\OT 2026 materials-20260820T135936Z-1-001\OT_github\OT1\BigM.py"
```

Add a device